TP – Tableau de bord de notes

Pr. Ahmed Bentajer

Objectifs

- Calculer une moyenne pondérée avec `reduce`
- Extraire une sous-liste avec `filter`, trier une liste avec `sort` sans muter l'original
- Enchaîner plusieurs `computed` qui se construisent les unes sur les autres
- Une chaîne `v-if`/`v-else-if`/`v-else-if`/`v-else-if`/`v-else` à 5 branches

Cahier des charges

Un tableau de bord qui calcule la moyenne pondérée d'un semestre à partir d'une liste de modules (note et coefficient), affiche une mention automatique, classe les modules du meilleur au moins bon, et signale ceux à rattraper.

Pour info

Ce TP introduit trois fonctions de tableau que vous n'avez pas encore utilisées : `reduce`, `filter` et `sort`. Elles remplacent les boucles `for...of` utilisées dans les TP précédents pour le même genre de calcul, mais à manier avec un peu plus de rigueu.

```
// reduce() condense un tableau en une seule valeur, en partant d'une valeur
// initiale (ici 0) et en l'accumulant element par element :
const total = [10, 20, 30].reduce((accumulateur, x) => accumulateur + x, 0)
// total vaut 60. A chaque etape, "accumulateur" recoit ce que la fonction a
// renvoye a l'etape precedente.

// filter() garde seulement les elements qui verifient une condition :
const pairs = [1, 2, 3, 4].filter(x => x % 2 === 0) // [2, 4]

// sort() trie EN PLACE (il modifie le tableau original) et renvoie ce meme
// tableau. Pour ne pas muter les donnees de l'application, on le fait
// toujours sur une COPIE :
const copie = [...tableau].sort((a, b) => a - b) // ordre croissant
```

1 Moyenne générale pondérée

Complétez `moyenneGenerale` : la moyenne pondérée de **tous** les modules est la somme des (note × coefficient) de chaque module, divisée par la somme des coefficients. Utilisez `reduce` pour chacune des deux sommes, puis arrondissez le résultat à 2 décimales.

Faites attention !!

`reduce` sans valeur initiale explicite (le deuxième argument, ici `0`) démarre son accumulation sur le **premier élément** du tableau plutôt que sur `0` – ça change le résultat, et ça plante carrément sur un tableau vide (`reduce` sans valeur initiale lève une erreur si le tableau est vide). Donnez toujours cette valeur initiale explicitement.

2 Mention automatique : 5 branches

À partir de `moyenneGenerale`, affichez une mention : Très bien ($\geq$ 16), Bien ($\geq$ 14), Assez bien ($\geq$ 12), Passable ($\geq$ 10), Insuffisant (en dessous). Une chaîne `v-if`/`v-else-if` (trois fois)/`v-else`.

3 Modules à rattraper

Complétez `modulesARattraper` avec `filter` : tous les modules de `this.modules` dont la note est strictement inférieure à 10.

4 Classement et recherche

Complétez `modulesTries` : une copie de `this.modules` triée par note décroissante. Complétez ensuite `modulesAffiches` : `modulesTries` filtré selon `this.recherche` (sur le nom, insensible à la casse) – c'est cette dernière liste qui est effectivement affichée dans le `v-for`.

Faites attention !!

`[a, b] => a - b` trie par ordre **croissant** ; pour un ordre décroissant, c'est `b - a`. C'est un détail facile à inverser par erreur – si votre classement affiche le plus faible en premier, c'est probablement ça.

5 Meilleur et moins bon module

Complétez `moduleMax` et `moduleMin`, en vous appuyant sur `modulesTries` (déjà trié – pas besoin de reparcourir ni retrier). Gérez le cas limite où `this.modules` serait vide.

Faites attention !!

Pourquoi se baser sur `modulesTries` et non sur `modulesAffiches` ? Parce que `modulesAffiches` dépend de la recherche en cours : si le meilleur module du semestre ne correspond pas au texte tapé dans la barre de recherche, il disparaîtrait à tort de « Meilleur module ». Un classement global ne doit pas dépendre d'un filtre d'affichage local.

6 Ajout, suppression, détail

Mêmes patterns que dans les deux TP précédents : bouclez sur `modulesAffiches`, complétez `ajouterModule()` et `supprimerModule(module)`, et branchez le `v-show` du panneau de détail sur `toggleDetail` (déjà fourni).

Aller plus loin

- `tousValides()` : renvoyez `true` si tous les modules ont une note $\geq$ 10, avec `Array.prototype.every`.
- Affichez le nombre de modules à rattraper directement dans le badge de la section correspondante, calculé avec `modulesARattraper.length`.